

PRUEBA DE EVALUACIÓN CONTINUA

María Soriano Palacios

1. Describa el funcionamiento de los protocolos de comunicación I2C/TWI y SPI en placas Arduino

¿Qué es I2C / TWI?

I2C (*Inter-Integrated Circuit*) o, como también se le conoce en Arduino (AVR), **TWI** (*Two-Wire Interface*) es un **protocolo de comunicación serie¹, síncrono² y maestro-esclavo³** que utiliza solo **dos líneas** o hilos: **SDA** (*Serial Data*): línea utilizada para transmitir datos y **SCL** (*Serial Clock*): línea utilizada para sincronizar la comunicación.

¿Cómo funciona?

Un dispositivo maestro, como Arduino, inicia la comunicación y genera la señal de reloj por SCL. La comunicación comienza con una condición de **inicio**, en la que el maestro baja la línea SDA mientras SCL permanece en alto. A continuación, se produce la fase de **direccionamiento**, donde el maestro envía la dirección del dispositivo esclavo con el que quiere comunicarse, junto con un bit que indica si la operación será de **lectura** o **escritura**.

Después, tiene lugar la fase de **transmisión de datos**, en la que se transmiten los datos byte a byte. Tras cada byte, el receptor (normalmente el esclavo) responde con un bit de **confirmación** llamado **ACK** (*Acknowledgement*) / **NACK** (*Not Acknowledgement*). Finalmente, el maestro genera una condición de **parada** y libera el bus, poniendo la línea SDA a nivel alto mientras SCL está en alto.

¹ Comunicación serie:

Es un tipo de comunicación en la que los datos se transmiten **bit a bit a través de una única línea de datos**, en lugar de enviar varios bits a la vez como en la comunicación paralela.

² Comunicación síncrona:

Es aquella en la que la transmisión de datos está **sincronizada mediante una señal de reloj común**. Este reloj indica cuándo deben leerse o enviarse los bits.

³Arquitectura maestro-esclavo:

Es un modelo en el que existe un dispositivo **maestro**, que controla la comunicación (inicia la transmisión y genera el reloj), y uno o varios dispositivos **esclavos**, que responden a las peticiones del maestro sin iniciar la comunicación por sí mismos.

Características

Entre sus características principales, destacamos que solo necesita **2 cables**, lo que simplifica el montaje y facilita la conexión de diferentes periféricos. Además, permite conectar **varios dispositivos** en el **mismo bus**. Sin embargo, este protocolo es **más lento que SPI**, su **comunicación** es más **compleja** y su **velocidad** y distancia están más **limitadas**.

En placas Arduino, la comunicación se implementa mediante la *librería wire.h*, utilizando pines específicos (como SDA y SCL).

¿Qué es SPI?

SPI (*Serial Peripheral Interface*) es también un **protocolo de comunicación serie, síncrono y maestro-esclavo**, pero a diferencia de I2C, usa 4 líneas o señales para una transmisión *full-dúplex*⁴: **MOSI** (*Master Out Slave In*): línea de datos del maestro al esclavo, **MISO** (*Master In Slave Out*): línea de datos del esclavo al maestro, **SCK** (*Serial Clock*): señal de reloj y **SS/CS** (*Slave Select*): línea de selección de esclavo.

¿Cómo funciona?

Un dispositivo maestro, como Arduino, inicia la comunicación y genera la señal de reloj a través de la línea SCK. A diferencia de I2C, no existe una primera **fase de direccionamiento por dirección**, en su lugar, el maestro selecciona directamente al dispositivo esclavo activando su línea correspondiente **SS/CS** (*Slave Select/Chip Select*), poniéndola a nivel bajo.

- ⁴ **Comunicación Full-Dúplex**: Permite enviar y recibir datos simultáneamente, permitiendo una comunicación bidireccional en tiempo real sin necesidad de alternar los canales de envío y recepción.

A continuación, comienza la fase de **transmisión de datos**, que se realiza de forma **simultánea** en ambos sentidos (*full dúplex*). El maestro envía datos al esclavo a través de la línea **MOSI**, mientras que el esclavo envía datos al maestro por la línea **MISO**, sincronizados con la señal de reloj SCK.

Los datos se **transmiten** bit a bit en cada ciclo de reloj, normalmente en grupos de **8** bits (bytes). A diferencia de I2C, SPI **no** utiliza bits de **confirmación** como *ACK/NACK*, ya que la comunicación es directa y controlada por el maestro.

Finalmente, el maestro termina la comunicación **desactivando** la línea SS/CS (poniéndola a nivel alto), indicando al esclavo la finalización de la transmisión.

Características

Aunque SPI es más rápido que I2C y ofrece una comunicación muy **eficiente**, no utiliza direcciones como I2C y requiere **más líneas de conexión**, ya que cada dispositivo esclavo necesita su propia línea SS/CS.

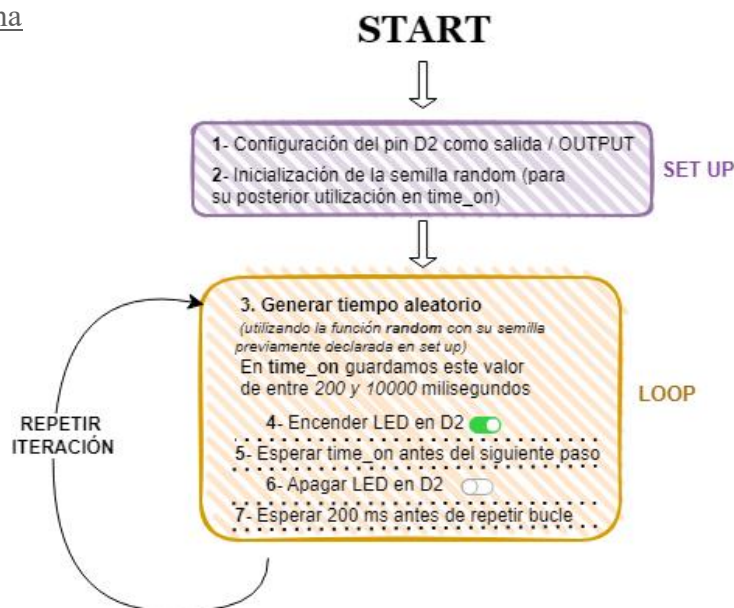
En Arduino, este protocolo se implementa mediante la *librería SPI.h*, utilizando pines específicos para MOSI, MISO, SCK y SS.

2. Diseñe un programa en C ++ para Arduino que encienda en un bucle infinito un LED ubicado en el pin D2. En cada iteración el LED se iluminará durante un período de tiempo aleatorio entre 200 ms y 10 segundos.

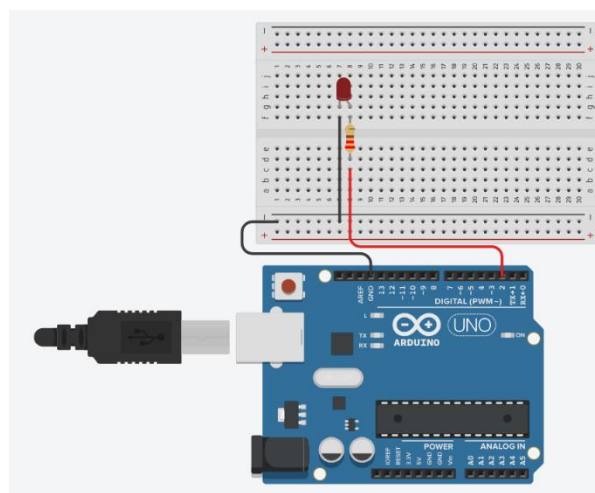
Explicación

Para resolver este problema, se configura el pin digital D2 como salida y se conecta el LED a dicho pin. Después, dentro del bucle principal del programa, se genera un tiempo aleatorio comprendido entre 200 milisegundos y 10000 milisegundos (10 segundos). Para obtener un valor aleatorio en el programa, hemos inicializado una semilla usando una entrada analógica desconectada. El LED se encenderá durante ese tiempo y luego se apagará. El proceso se repite indefinidamente esperando 200 milisegundos cada vez.

Diagrama



Diseño de montaje



Código

```
1  #include <Arduino.h> // Includes basic Arduino functions and defs
2
3  const int LED_PIN = 2; // Constant storing pin number where LED will be connected
4
5  // The setup() function runs once when the board starts
6  void setup()
7  {
8      pinMode(LED_PIN, OUTPUT); // Configures LED pin as a digital output
9      randomSeed(analogRead(A0)); // Initializes random seed using noise from an unconnected analog pin
10 }
11
12 // The loop() function runs repeatedly in an infinite loop
13 void loop()
14 {
15     long time_on = random(200, 10001); // Generates random time between 200 - 10000 ms
16     digitalWrite(LED_PIN, HIGH); // Turns LED ON by setting the pin to HIGH
17     delay(time_on); // Keeps LED ON for time_on ms
18     digitalWrite(LED_PIN, LOW); // Turns LED OFF by setting the pin to LOW
19     delay(200); // Waits 200 ms before next iteration
20 }
--
```

3. Diseñe un programa en Python para Raspberry Pi que utilice 5 pines de salida GPIO y 1 pin de entrada digital.

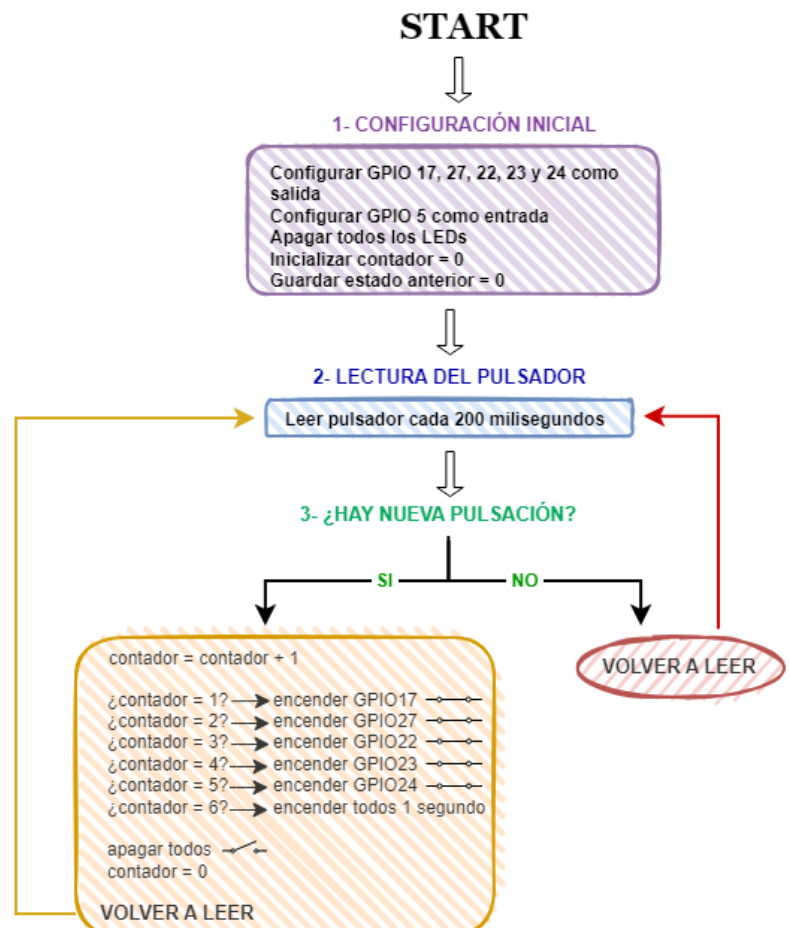
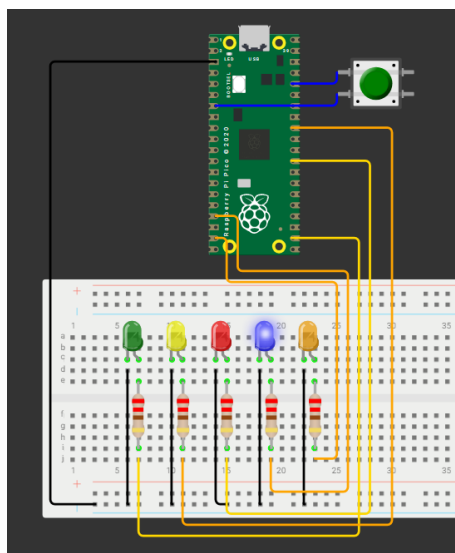
- Los 5 LEDs estarán conectados a los pines GPIO 17, 27, 22, 23 y 24.
- Un pulsador estará conectado al pin GPIO 5, configurado como entrada digital.

Explicación

Para resolver este problema, se configuran cinco pines GPIO como salida para controlar los LEDs y un pin GPIO como entrada digital para leer el pulsador. El programa realiza una lectura periódica cada 200 milisegundos y detecta cuándo se produce una nueva pulsación. Para ello, se utiliza un contador de pulsaciones acumuladas. Según el valor de ese contador, se enciende uno de los LEDs correspondientes. Cuando se llega a la sexta pulsación, se encienden todos los LEDs durante 1 segundo, se apagan y el contador vuelve a cero para reiniciar el ciclo.

Además, para que el sistema termine de forma segura, al salir del programa se apagan los LEDs y se liberan los GPIO.

Diseño y Diagrama



Código

```

1 import RPi.GPIO as GPIO # Import library to control Raspberry Pi GPIO pins
2 import time              # Import time library to handle delays
3
4 led_pins = [17, 27, 22, 23, 24] # List of GPIO pins where LEDs will be connected
5 button_pin = 5                  # GPIO pin where push button will be connected
6 press_count = 0                 # Counter to store number of button presses
7 previous_state = 0             # Variable to store previous button state for edge detection
8                                 # edge detection is used to detect changes in a signal
9
10 GPIO.setmode(GPIO.BCM) # Uses BCM numbering (GPIO numbers instead of physical pin numbers)
11 GPIO.setwarnings(False) # Disables GPIO warnings !
12
13 for led in led_pins:          # LOOP through all LED pins
14     GPIO.setup(led, GPIO.OUT)  # Configures each LED pin as output
15     GPIO.output(led, GPIO.LOW) # Ensures all LEDs start turned OFF
16
17 # Configures button as input with pull-down resistor
18 GPIO.setup(button_pin, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
19
20 def turn_off_all():           # FUNCTION to TURN OFF all LEDs
21     for led in led_pins:      # Loops through all LED pins
22         GPIO.output(led, GPIO.LOW) # Sets each LED to LOW (OFF)
23
24 def turn_on_one(index):       # FUNCTION to TURN ON LED by index
25     turn_off_all()            # Turns OFF all LEDs
26     GPIO.output(led_pins[index], GPIO.HIGH) # Sets selected by index LED to HIGH (ON)
27
28 try:
29     while true:                # INFINITE LOOP
30         current_state = GPIO.input(button_pin) # Reads current state of button
31
32         if current_state == 1 and previous_state == 0: # Detects rising edge (LOW (0) → HIGH (1))
33             press_count += 1 # If so, increases press counter
34             if press_count == 1: # If press counter equals 1 (one pulsation)
35                 turn_on_one(0) # Turns ON LED at GPIO17
36             elif press_count == 2: # If press counter equals 2 (two pulsations)
37                 turn_on_one(1) # Turns ON LED at GPIO27
38             elif press_count == 3: # If press counter equals 3 (three pulsations)
39                 turn_on_one(2) # Turns ON LED at GPIO22
40             elif press_count == 4: # If press counter equals 4 (four pulsations)
41                 turn_on_one(3) # Turns ON LED at GPIO23
42             elif press_count == 5: # If press counter equals 5 (five pulsations)
43                 turn_on_one(4) # Turns ON LED at GPIO24
44             elif press_count == 6: # If press counter equals 6 (six pulsations)
45                 for led in led_pins: # Turns ON all LEDs
46                     GPIO.output(led, GPIO.HIGH)
47                 time.sleep(1) # Keeps all LEDs ON for 1 second
48                 turn_off_all() # Turns OFF all LEDs
49                 press_count = 0 # Resets counter
50             previous_state = current_state # Updates previous state to current
51             time.sleep(0.2) # Waits 200 ms
52
53 except KeyboardInterrupt:
54     turn_off_all() # IMPORTANT! This handles manual interruption (Ctrl+C)
55     GPIO.cleanup() # Turns OFF all LEDs before exiting
                    # Resets GPIO configuration to a safe state

```